



Projet de Fin d'Année

Présenté à

École Nationale d'Ingénieurs de Sfax (ENIS)
Génie Informatique

Réalisé par

Yessmine BENHLIMA
Mahmoud ELLOUMI
Oumayma KAMMOUN

Application de traitement intelligent de documents commerciaux

Encadré par :

Mr. Riadh BEN HALIMA

Soutenu le **02 mai 2026**

devant le jury composé de :

Mr. Tarek FRIKHA



Remerciements

Nous remercions sincèrement notre encadrant, Monsieur **Riadh Ben Halima**, pour ses précieux conseils et son soutien tout au long de ce projet. Sa disponibilité et ses retours constructifs ont été essentiels à l'avancement de notre travail.

Nous tenons également à exprimer notre gratitude à Monsieur **Tarak Frikha** pour sa participation en tant que membre du jury et pour ses remarques enrichissantes lors de l'évaluation de notre projet.

Nous remercions également l'ensemble du corps enseignant et administratif du Département Génie Informatique et de Mathématiques Appliquées de l'ENIS pour leur soutien.

Enfin, un grand merci à nos familles et amis pour leur soutien moral.

Yessmine Benhlima
Oumayma Kammoun
Mahmoud Elloumi



Résumé

Ce projet académique présente la conception et le développement de **Sparky**, une application web dédiée au traitement intelligent de documents commerciaux. Dans un contexte de digitalisation des processus métiers, elle répond au besoin d'automatiser l'extraction et le traitement des informations contenues dans divers documents tels que factures, bons de livraison, bons de commande, avoirs et devis.

L'application repose sur une architecture moderne en trois tiers : un frontend **Angular**, un backend Python **FastAPI** exposant une API REST asynchrone intégrant un moteur d'intelligence artificielle basé sur **Mistral AI** pour l'OCR intelligent et l'extraction structurée des données, et une couche de persistance **PostgreSQL**. Le middleware **n8n** intervient en tant que composant transversal d'orchestration des workflows, sans constituer un tier applicatif à part entière.

Les fonctionnalités principales incluent : la numérisation et la classification automatique des documents, la gestion et le filtrage des fichiers, un comparateur de prix multi-fournisseurs, ainsi qu'un tableau de bord analytique.

Enfin, ce rapport compare une première solution prototype basée sur n8n et Ollama (Qwen2.5-B7) en local avec la solution finale retenue, en mettant en évidence les critères de performance, sécurité et maintenabilité ayant guidé ce choix.



Table des matières

Remerciements	i
Résumé	ii
Liste des figures	vi
Liste des acronymes	vii
Introduction Générale	1
1 Contexte général et analyse des besoins	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	4
1.3 Problématique	4
1.4 Étude de l'existant et critique	5
1.4.1 Approches traditionnelles	5
1.4.2 Solutions existantes sur le marché	5
1.4.3 Synthèse critique	7
1.5 Objectifs du projet	7
1.6 Solution proposée	8
1.7 Conclusion	9
2 Analyse et Conception	10
2.1 Introduction	10
2.2 Méthodologie Agile	10
2.2.1 Scrum	10

2.2.2	Planification des sprints	11
2.2.3	Backlog du produit	11
2.3	Analyse des besoins	12
2.3.1	Besoins fonctionnels	12
2.3.2	Besoins non fonctionnels	13
2.4	Architecture globale	14
2.5	Conception de la base de données	15
2.6	Diagramme des cas d'utilisation	16
2.7	Diagramme d'activité	17
2.8	Diagramme de classes	18
2.9	Diagrammes de séquence	19
2.9.1	Processus de scan et extraction	19
2.9.2	Processus de comparaison de prix	21
2.10	Choix technologiques	22
2.10.1	Frontend : Angular 20	22
2.10.2	Backend : FastAPI	22
2.10.3	IA : Mistral AI	23
2.10.4	Base de données : PostgreSQL	23
2.10.5	Middleware : n8n	24
2.11	Conclusion	24
3	Développement et Implémentation	25
3.1	Introduction	25
3.2	Environnement de développement	25
3.2.1	Utilisation de Docker	26
3.3	Développement du backend	27
3.3.1	Architecture générale	27
3.3.2	Couche métier (Services)	27
3.3.3	Pipeline d'extraction IA	28
3.3.4	Diagramme d'États — Cycle de Vie d'un Document	29
3.4	Développement du frontend	32
3.4.1	Architecture du Frontend	32
3.4.2	Gestion des services	32
3.4.3	Gestion de l'état	33
3.4.4	Interface utilisateur	33
3.5	Middleware n8n	33

3.5.1	Description du workflow n8n	34
3.5.2	Exemple de notification email	36
3.6	Première solution : n8n + Ollama Qwen2.5-B7	38
3.6.1	Avantages	38
3.6.2	Limites	38
3.7	Comparaison des solutions	39
3.8	Choix final et justification	40
3.9	Conclusion	40
4	Présentation de l'Application	41
4.1	Introduction	41
4.2	Page de connexion	41
4.3	Module Tableau de bord	42
4.4	Module Scanner de documents	43
4.5	Module Comparateur de prix	45
4.6	Matching IA + PostgreSQL	46
4.6.1	Stratégie de matching en cascade	46
4.6.2	Détection de catégorie	47
4.6.3	Stockage des données	47
4.6.4	Impact fonctionnel	47
4.7	Module Gestion des documents	49
4.8	Conclusion	50



Table des figures

1	Logo de Sparky Solutions Photovoltaïques	4
2	Architecture 3-tiers du système Sparky	15
3	Diagramme des cas d'utilisation de l'application SPARKY	17
4	Diagramme d'activité du système SPARKY	18
5	Diagramme de classes du système Sparky	19
6	Diagramme de séquence - Scan et extraction	20
7	Diagramme de séquence - Comparaison de prix	21
8	Logo Angular	22
9	Logo Fast API	23
10	Logo Mistral AI	23
11	Logo PostgreSQL	24
12	Logo N8N	24
13	Pipeline d'extraction IA en cascade	29
14	Diagramme d'états du cycle de vie d'un document	30
15	Conception et exécution d'un workflow avec n8n	35
16	Exemple de notification email générée par le workflow SPARKY	37
17	Exemple de données structurées dans Google Sheets	38
18	Interface de la page de connexion	42
19	Interface du module Tableau de bord	43
20	Interface du module Scanner de documents	43
21	Présentation structurée des informations extraites à partir d'un devis	44
22	Interface du module Comparateur de prix	46
23	Interface du module Gestion des documents	49



Liste des acronymes

OCR : Optical Character Recognition

IA : Intelligence Artificielle

LLM : Large Language Model

API : Application Programming Interface

JSON : JavaScript Object Notation

REST : Representational State Transfer

NLP : Natural Language Processing

MVC : Model View Controller

UI : User Interface

DB : Base de Données

SaaS : Software as a Service



Introduction Générale

Dans un monde où la digitalisation transforme en profondeur les modes de fonctionnement des entreprises, la gestion documentaire constitue l'un des domaines les plus impactés par les avancées technologiques. Les documents commerciaux - factures, bons de livraison, bons de commande, avoirs et devis - représentent des flux informationnels critiques qui conditionnent la qualité des processus comptables, logistiques et commerciaux. Pourtant, de nombreuses organisations continuent de traiter ces documents manuellement, générant des coûts importants, des délais de traitement prohibitifs et des taux d'erreur non négligeables.

L'intelligence artificielle, et particulièrement les modèles de vision par ordinateur couplés aux capacités de traitement du langage naturel, offre aujourd'hui des perspectives révolutionnaires pour l'automatisation de ces tâches répétitives. La reconnaissance optique de caractères (OCR) intelligente permet non seulement de numériser les documents, mais surtout d'en comprendre la sémantique et d'en extraire des données structurées exploitables directement par les systèmes d'information.

C'est dans ce contexte que s'inscrit le projet Sparky, une application web complète dédiée au traitement intelligent des documents commerciaux. Ce projet vise à démontrer la faisabilité technique et la valeur ajoutée opérationnelle d'une solution intégrant les technologies les plus récentes en matière d'IA, de développement web et d'architecture distribuée. Le présent rapport décrit l'intégralité du cycle de vie du projet, depuis l'analyse des besoins jusqu'à la présentation des fonctionnalités déployées.

Il est structure en cinq chapitres principaux. Le premier chapitre pose le contexte et la problematique. Le second detaille l'analyse et la specification des besoins. Le troisieme presente la conception architecturale avec les diagrammes UML. Le quatrieme traite du developpement et de l'implementation, en incluant une analyse comparative entre la solution prototype et la solution finale. Le cinquieme chapitre presente l'application deployee avec ses differentes interfaces.

A travers ce travail, nous demontrons comment l'integration de technologies modernes - Angular 20, FastAPI, Mistral AI, PostgreSQL, Docker et n8n - permet de construire une solution robuste, evolutive et performante, capable de repondre aux exigences du traitement documentaire en milieu professionnel.

Chapitre

1

Contexte général et analyse des besoins

1.1 *Introduction*

Ce premier chapitre présente le cadre général du projet SPARKY. Il introduit le contexte dans lequel s'inscrit le travail, la problématique étudiée ainsi que les objectifs visés et la solution proposée. Cette étape permet de situer le projet dans le cadre plus large de la transformation numérique des processus de gestion documentaire et de l'exploitation intelligente des données.

Par ailleurs, la réussite d'un projet informatique repose essentiellement sur une analyse rigoureuse de l'existant et des solutions disponibles. Cette étape constitue un support essentiel aux choix de conception et de développement.

Dans ce sens, ce chapitre présente une étude de l'existant, suivie d'une analyse critique des solutions actuelles et d'une proposition de solution répondant aux limites identifiées.

1.2 *Présentation de l'organisme d'accueil*



FIGURE 1. *Logo de Sparky Solutions Photovoltaïques*

Le projet **SPARKY** a été réalisé dans un cadre académique en collaboration avec l'entreprise **Sparky Solutions Photovoltaïques**, société d'ingénierie basée à Sfax et spécialisée dans les solutions énergétiques photovoltaïques.

L'entreprise accompagne les particuliers et les professionnels dans leur transition vers une énergie propre et durable en proposant des services couvrant l'ensemble du cycle de vie des projets photovoltaïques : étude, conception, installation, mise en service et maintenance.

Grâce à son expertise technique et à son engagement envers l'innovation durable, Sparky Solutions Photovoltaïques offre un environnement favorable au développement de solutions intelligentes intégrant les technologies d'intelligence artificielle et de gestion documentaire.

C'est dans cette dynamique d'innovation et d'optimisation des processus métiers que le projet **SPARKY** a été conçu afin d'automatiser le traitement documentaire et d'améliorer l'exploitation des données commerciales.

1.3 *Problématique*

Malgré l'évolution des technologies numériques, la gestion des documents commerciaux demeure largement manuelle dans de nombreuses entreprises. Cette situation engendre plusieurs contraintes :

- un temps important consacré au traitement des documents ;
- des erreurs humaines lors de la saisie des données ;

- un manque de centralisation et de traçabilité des informations ;
- une difficulté d'exploitation des données pour l'aide à la décision.

Comment concevoir une solution intelligente capable d'automatiser le traitement, l'extraction et l'exploitation des documents commerciaux tout en garantissant précision, performance et simplicité d'utilisation ?

Cette problématique soulève également plusieurs défis liés au choix des technologies d'intelligence artificielle, à l'architecture logicielle et à la sécurisation des données sensibles.

1.4 Étude de l'existant et critique

1.4.1 Approches traditionnelles

Les approches traditionnelles reposent essentiellement sur une gestion manuelle des documents commerciaux. Les informations présentes dans les factures, devis ou bons de livraison sont généralement saisies manuellement dans les systèmes ERP ou comptables.

Cette méthode présente plusieurs limites majeures :

- Temps de traitement élevé ;
- Risque important d'erreurs humaines ;
- Difficulté de recherche et d'archivage ;
- Faible productivité ;
- Exploitation limitée des données.

1.4.2 Solutions existantes sur le marché

Le Tableau 1 présente une comparaison des principales solutions existantes en matière d'extraction et de gestion intelligente de documents.

TABLEAU 1. *Comparaison des solutions existantes*

Solution	Description	Limites / Critiques
Google Document AI	Plateforme cloud d'extraction intelligente basée sur le machine learning.	— Coûts élevés à grande échelle — Dépendance au cloud — Problèmes de confidentialité — Nécessite une connexion stable
AWS Textract	Service AWS d'extraction automatique de texte et de données.	— Coûts variables selon le volume — Intégration complexe hors AWS — Risque de vendor lock-in — Contraintes RGPD
Adobe Acrobat Pro	Solution OCR intégrée pour les documents PDF.	— Extraction limitée au texte brut — Pas de structuration intelligente — Absence d'analyse métier
ABBYY FineReader	Logiciel OCR professionnel reconnu pour sa précision.	— Licence coûteuse — Fonctionnalités IA limitées — Interface peu intuitive
DocuWare / M-Files	Solutions de gestion électronique de documents (GED).	— Déploiement complexe — Coûts élevés — Solutions souvent surdimensionnées
Tesseract OCR	Moteur OCR open-source développé par Google.	— Nécessite un développement important — Absence de compréhension contextuelle — Pas d'interface native

1.4.3 Synthèse critique

L'étude de l'existant montre qu'aucune solution ne répond parfaitement aux besoins des PME souhaitant disposer d'un système :

- précis dans l'extraction des données ;
- performant et rapide ;
- flexible et évolutif ;
- sécurisé et maîtrisé localement ;
- économiquement accessible.

Les solutions cloud offrent de bonnes performances mais impliquent des contraintes de coût, de dépendance et de confidentialité. À l'inverse, les solutions open-source locales offrent davantage de contrôle mais restent limitées en matière d'intelligence documentaire et d'expérience utilisateur.

Cette analyse justifie le développement d'une solution hybride intelligente capable de combiner les avantages des solutions cloud et des approches locales tout en minimisant leurs limites.

1.5 Objectifs du projet

Les objectifs du projet **SPARKY** ne se limitent pas à la création d'une simple application logicielle. Ils visent principalement à répondre à des problématiques opérationnelles réelles liées à la gestion et au traitement des documents commerciaux au sein des entreprises.

Les principaux objectifs du projet sont les suivants :

- **Accélérer le processus de traitement documentaire** en réduisant considérablement le temps consacré à la saisie manuelle des informations issues des documents commerciaux ;
- **Optimiser la productivité des équipes administratives et commerciales** en leur permettant de se concentrer sur des tâches à plus forte valeur ajoutée ;

- **Réduire les erreurs humaines** liées à la transcription manuelle des données vers les systèmes d'information ;
- **Améliorer la traçabilité et la centralisation des données** afin de garantir une meilleure organisation et accessibilité des informations commerciales ;
- **Faciliter la prise de décision** grâce à l'exploitation intelligente et automatisée des données extraites, notamment à travers la comparaison des prix fournisseurs et les analyses statistiques.

Le projet SPARKY vise à transformer les documents commerciaux non structurés en données exploitables afin d'améliorer l'efficacité opérationnelle, la fiabilité des traitements et la qualité des décisions stratégiques.

1.6 *Solution proposée*

Afin de répondre aux besoins identifiés et d'atteindre les objectifs fixés, nous proposons le développement d'une application logicielle intelligente dédiée au traitement automatisé des documents commerciaux.

La solution proposée permet notamment de :

- **Automatiser la numérisation et l'import des documents** (PDF, images) via une interface web intuitive et ergonomique ;
- **Classifier automatiquement les documents** tels que les factures, devis et bons de livraison grâce à des algorithmes d'intelligence artificielle ;
- **Extraire de manière structurée les informations essentielles** comme les montants, dates, fournisseurs et articles avec un haut niveau de précision ;
- **Centraliser la gestion documentaire** dans une base de données sécurisée intégrant des fonctionnalités avancées de recherche et de filtrage ;
- **Comparer automatiquement les prix entre fournisseurs** afin d'optimiser les achats et améliorer les stratégies de négociation commerciale ;

- **Fournir un tableau de bord analytique** intégrant des indicateurs de performance, des statistiques et des visualisations de données pour l'aide à la décision.

La solution SPARKY combine automatisation intelligente, sécurité des données et assistance décisionnelle afin d'offrir une plateforme moderne capable d'optimiser durablement les processus documentaires des entreprises.

1.7 Conclusion

Ce chapitre a permis de présenter le cadre général du projet SPARKY ainsi que les motivations ayant conduit à sa réalisation. Il a mis en évidence la problématique liée à la gestion manuelle des documents commerciaux et les limites des solutions existantes.

L'étude de l'existant a permis de dégager les insuffisances des approches traditionnelles et des solutions actuelles, justifiant ainsi le besoin d'un système intelligent, automatisé et sécurisé.

Enfin, une solution globale a été proposée afin de répondre aux contraintes identifiées et d'améliorer l'efficacité du traitement des documents commerciaux.

Chapitre

2

Analyse et Conception

2.1 Introduction

La phase de conception constitue le pont entre l'analyse des besoins et l'implémentation technique. C'est durant cette étape que les choix architecturaux structurants sont réalisés, influençant durablement la qualité, la performance et la maintenabilité de l'application.

Ce chapitre présente la conception architecturale du système Sparky, en détaillant l'architecture globale en trois tiers, la modélisation de la base de données, les diagrammes de classes et de séquence, ainsi que les justifications des choix technologiques.

2.2 Méthodologie Agile

2.2.1 Scrum

Dans le cadre du projet SPARKY, la méthodologie agile Scrum a été adoptée afin d'organiser le développement de manière itérative et incrémentale. Cette approche permet une meilleure adaptation aux besoins métier, une priorisation continue des fonctionnalités ainsi qu'une livraison progressive des différentes composantes du système.

Le projet a été structuré autour de plusieurs sprints intégrant les phases de planification, développement, tests et validation. Les réunions de suivi ont permis d'évaluer l'avancement des tâches et d'ajuster les priorités selon les besoins du projet.

2.2.2 Planification des sprints

Le tableau 2 montre les fonctionnalités du système qui ont été réparties sur plusieurs sprints afin d'assurer une progression cohérente et incrémentale du développement.

TABLEAU 2. *Planification des sprints*

Sprint	Durée	Objectif	User Stories
Sprint 0	1 semaine	Mise en place de l'environnement technique	Angular 20, FastAPI, PostgreSQL et n8n
Sprint 1	2 semaines	Authentification et base de données	US-01
Sprint 2	2 semaines	Import et classification des documents	US-02, US-03
Sprint 3	2 semaines	Extraction des données	US-04
Sprint 4	2 semaines	Évaluation et validation IA	US-05
Sprint 5	2 semaines	Analyse et tableau de bord	US-06, US-10
Sprint 6	2 semaines	Gestion catalogue et stock	US-07
Sprint 7	2 semaines	Export et automatisation	US-08, US-09
Sprint 8	1 semaine	Tests et validation finale	Tests + documentation

2.2.3 Backlog du produit

Le backlog du produit regroupe l'ensemble des fonctionnalités attendues du système SPARKY, classées par ordre de priorité selon la valeur apportée à l'utilisateur final dans le contexte de la gestion documentaire commerciale. Comme présenté dans le tableau 3, les fonctionnalités principales concernent l'importation et le traitement automatique des documents, l'extraction des données structurées, la validation des résultats, ainsi que l'analyse et l'export des informations extraites.

TABLE 3. *Backlog du produit du système SPARKY*

ID	User Story	Priorité	Estimation
US-01	En tant qu'utilisateur, je veux m'authentifier de manière sécurisée afin de protéger les données sensibles.	Haute	3 jours
US-02	En tant qu'utilisateur, je veux importer un document (PDF, JPG, PNG, TIFF) afin qu'il soit traité automatiquement.	Haute	3 jours
US-03	En tant qu'utilisateur, je veux que le système classe automatiquement les documents via l'IA.	Haute	5 jours
US-04	En tant qu'utilisateur, je veux extraire automatiquement les informations essentielles des documents.	Haute	6 jours
US-05	En tant qu'utilisateur, je veux visualiser le score de confiance associé aux résultats d'extraction.	Moyenne	2 jours
US-06	En tant qu'utilisateur, je veux comparer les prix entre plusieurs fournisseurs.	Moyenne	4 jours
US-07	En tant qu'utilisateur, je veux consulter le catalogue des articles et les mouvements de stock.	Moyenne	5 jours
US-08	En tant qu'utilisateur, je veux exporter les données extraites vers Excel et Google Sheets.	Moyenne	3 jours
US-09	En tant qu'utilisateur, je veux recevoir des notifications automatiques par email après traitement.	Moyenne	2 jours
US-10	En tant qu'utilisateur, je veux consulter un tableau de bord analytique des documents traités.	Haute	4 jours

2.3 Analyse des besoins

2.3.1 Besoins fonctionnels

Les besoins fonctionnels définissent l'ensemble des services et fonctionnalités que le système doit offrir aux utilisateurs, indépendamment des contraintes techniques ou des considérations liées à l'interface utilisateur, comme l'illustre le Tableau 4.

TABLEAU 4. *Besoins fonctionnels du système SPARKY*

Intitulé	Description	Priorité
Import et stockage des documents	Import de fichiers PDF/images, archivage et recherche	Haute
Classification automatique	Détection du type de document avec score de confiance	Haute
Extraction structurée	Extraction des informations clés (fournisseur, montants, articles)	Haute
Validation des données	Correction manuelle et validation avant enregistrement	Haute
Comparaison des prix	Analyse multi-fournisseurs et écarts de prix	Moyenne
Analyse et aide à la décision	Indicateurs, rapports et visualisations	Moyenne
Gestion des accès	Authentification et gestion du profil utilisateur	Moyenne
Export des données	Export des données et rapports en formats Excel / CSV	Moyenne

2.3.2 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les exigences qualitatives du système, notamment en termes de performance, de sécurité, de fiabilité et d'utilisabilité. Comme l'illustre le Tableau 5, ces exigences visent à garantir la qualité globale et la robustesse de la solution SPARKY.

TABEAU 5. *Besoins non fonctionnels du système SPARKY*

Critère	Exigence	Mesure / Indicateur
Performance	Temps de réponse pour le scan et l'extraction d'un document	$\leq 15s$ par document ; comparateur $\leq 60s$ pour 3 fournisseurs et 10 articles (contre 20 min en traitement manuel)
Ergonomie	Interface simple et intuitive	Actions principales accessibles en ≤ 3 clics
Fiabilité	Extraction fiable des données	Taux d'erreur $\leq 1\%$ sur documents lisibles ; validation si score $< 70\%$
Maintenabilité	Code structuré et testable	Code structuré en modules indépendants, services découplés, architecture testable
Portabilité	Compatibilité multi-navigateurs	Fonctionnement sur Chrome, Firefox, Edge et Safari
Extensibilité	Architecture évolutive	Ajout de nouveaux modules sans modification majeure du système

2.4 Architecture globale

La Figure 2 illustre l'architecture de Sparky qui repose sur un modèle en trois tiers (3-tier architecture), qui sépare clairement l'interface utilisateur, la couche applicative et la couche de données, afin d'assurer la modularité, la maintenabilité et l'évolutivité du système.

Les trois tiers principaux sont :

- **Tier 1 — Présentation** : frontend Angular 20, gérant l'interface utilisateur et les interactions.
- **Tier 2 — Logique applicative** : backend Python FastAPI, prenant en charge les traitements métier, l'intelligence artificielle et la génération des exports.
- **Tier 3 — Données** : PostgreSQL, assurant la persistance structurée et sécurisée des informations.

Le middleware **n8n** intervient en tant que composant transversal d'orchestration des workflows.

Il opère horizontalement entre le tier applicatif et les services externes (Google Sheets, messagerie SMTP), sans constituer un tier applicatif à part entière.

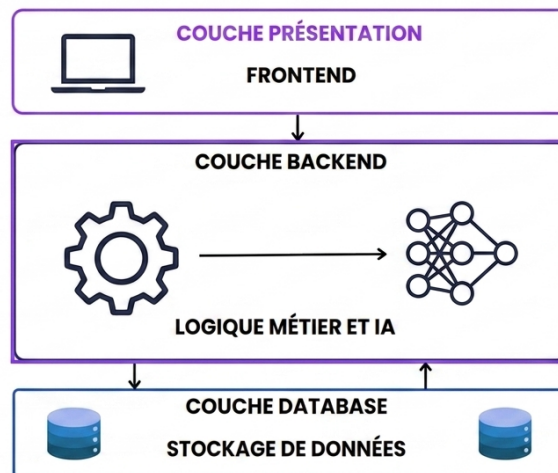


FIGURE 2. Architecture 3-tiers du système Sparky

2.5 Conception de la base de données

La base de données adoptée dans le système repose sur un modèle relationnel structuré autour des principales entités du domaine métier, tel qu'illustré dans le Tableau 6.

L'entité *Document* constitue l'élément central du modèle et regroupe les informations essentielles liées aux documents numérisés, notamment leur type, le fournisseur associé, les montants, le score de confiance ainsi que le statut de traitement.

L'entité *DocumentItem* représente les lignes d'articles extraites de chaque document, permettant une description détaillée de leur contenu.

L'entité *Supplier* regroupe les informations relatives aux fournisseurs du système, tandis que l'entité *ExtractionLog* assure la traçabilité des traitements d'intelligence artificielle appliqués aux documents.

Les relations entre les entités sont définies comme suit :

- Un *Document* peut contenir plusieurs *DocumentItem* (1-N).
- Un *Supplier* peut être associé à plusieurs *Documents* (1-N).
- Un *Document* peut générer plusieurs *ExtractionLog* (1-N).

TABLEAU 6. *Structure relationnelle de la base de données*

Table	Description	Clés
documents	Documents numérisés	id (PK), supplier_id (FK)
document_items	Articles des documents	id (PK), document_id (FK)
suppliers	Fournisseurs	id (PK)
extraction_logs	Logs des traitements IA	id (PK), document_id (FK)
price_comparisons	Comparaisons de prix	id (PK), document_id (FK)

2.6 Diagramme des cas d'utilisation

Le diagramme de cas d'utilisation est un outil de modélisation UML permettant de représenter les interactions entre les acteurs externes et le système. Il facilite la compréhension globale des fonctionnalités et valide la couverture des besoins fonctionnels.

Dans le cadre du projet, la figure 3 illustre le diagramme de cas d'utilisation du système SPARKY, mettant en évidence les interactions entre l'utilisateur et les principales fonctionnalités offertes par l'application.

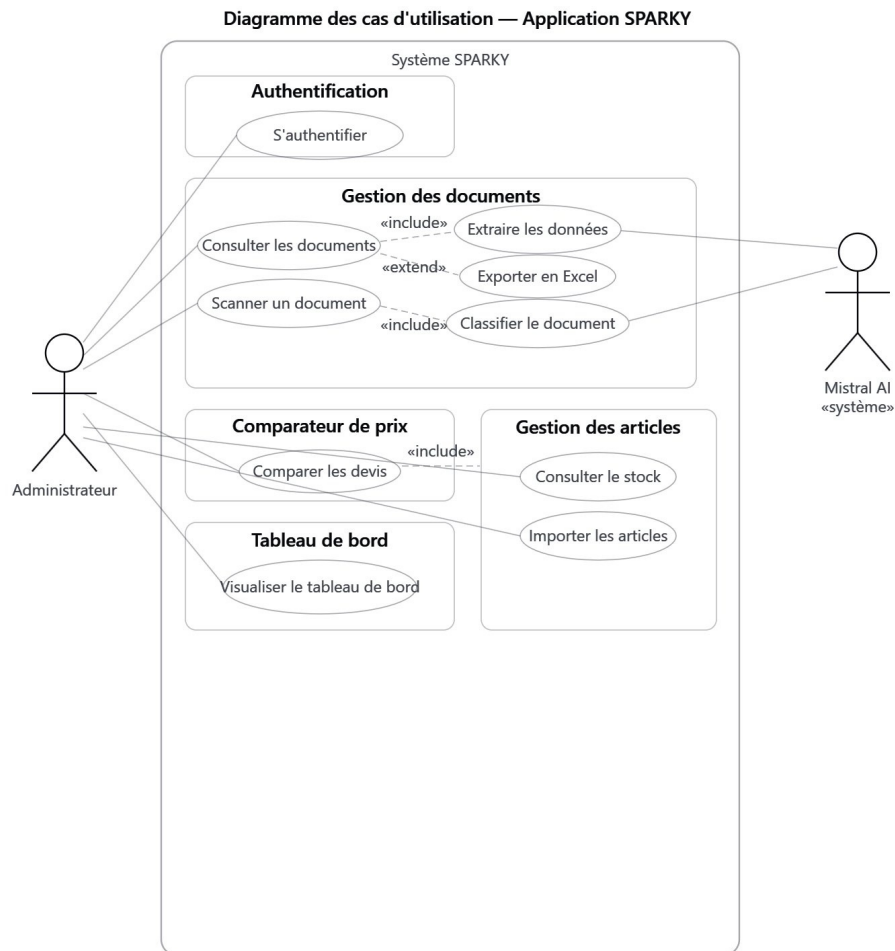


FIGURE 3. Diagramme des cas d'utilisation de l'application SPARKY

2.7 Diagramme d'activité

La figure 4 présente le diagramme d'activité du système SPARKY. Le processus débute par l'authentification de l'utilisateur, suivie du téléversement d'un document. L'application effectue ensuite la classification et l'extraction des données à l'aide de Mistral AI, avant l'enregistrement des résultats dans PostgreSQL. Une notification est ensuite envoyée à n8n pour automatiser certaines tâches, comme la génération de rapports et l'envoi d'emails. Enfin, l'utilisateur peut consulter, comparer ou exporter les résultats traités.

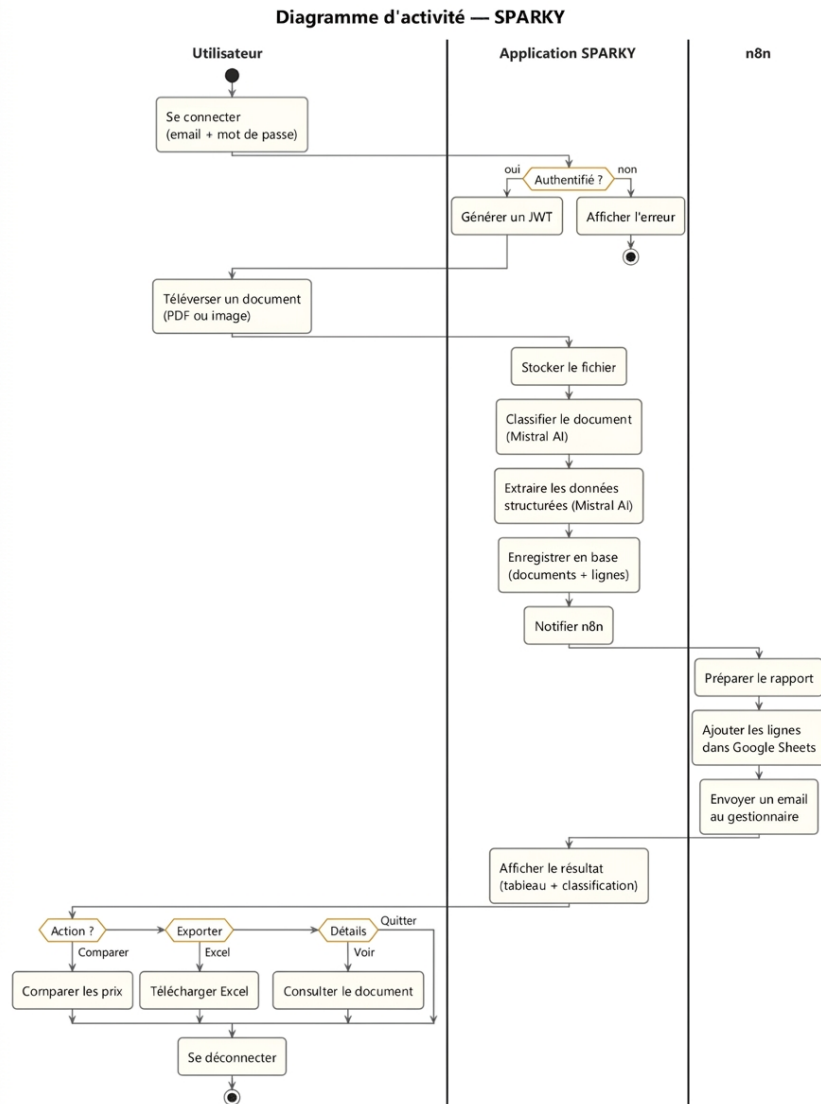


FIGURE 4. *Diagramme d'activité du système SPARKY*

2.8 Diagramme de classes

Le diagramme de classes de l'application Sparky modélise l'ensemble des entités du système ainsi que leurs relations, en mettant au centre la classe abstraite Document, qui constitue le noyau de l'architecture métier.

Cette figure 5 illustre le diagramme de classes du système Sparky..

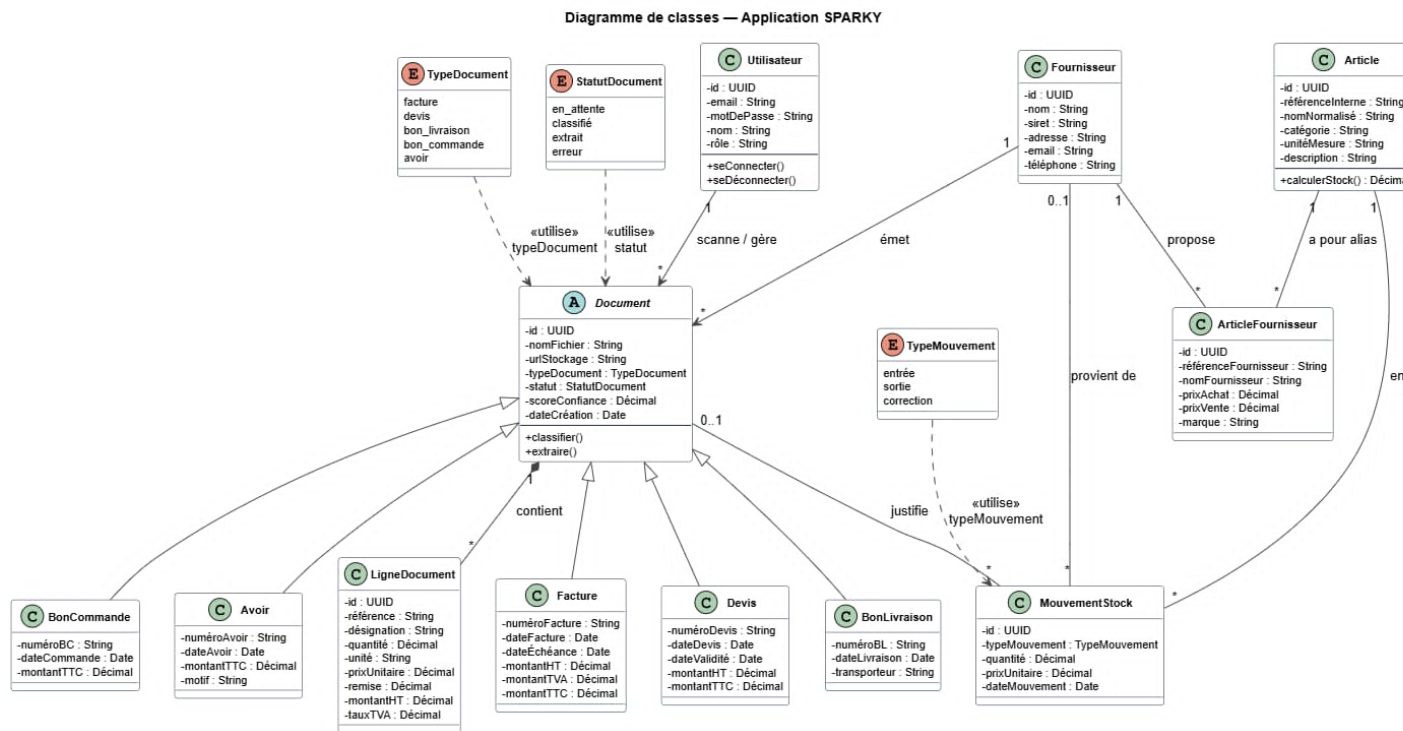


FIGURE 5. Diagramme de classes du système Sparky

2.9 Diagrammes de séquence

Les diagrammes de séquence UML décrivent les interactions dynamiques entre les objets dans le temps. Ils sont particulièrement utiles pour comprendre les flux de traitement complexes et identifier les points de synchronisation ou d'asynchronisme. Nous présentons deux diagrammes de séquence représentatifs du fonctionnement de Sparky.

2.9.1 Processus de scan et extraction

L'utilisateur télécharge un document via l'interface Angular. Une requête est envoyée à l'API FastAPI, qui transmet le document à Mistral AI pour traitement. Les résultats sont stockés dans PostgreSQL puis retournés au frontend.

Cette figure 6 met en évidence le diagramme de séquence décrivant le déroulement du processus de scan et d'extraction de documents.

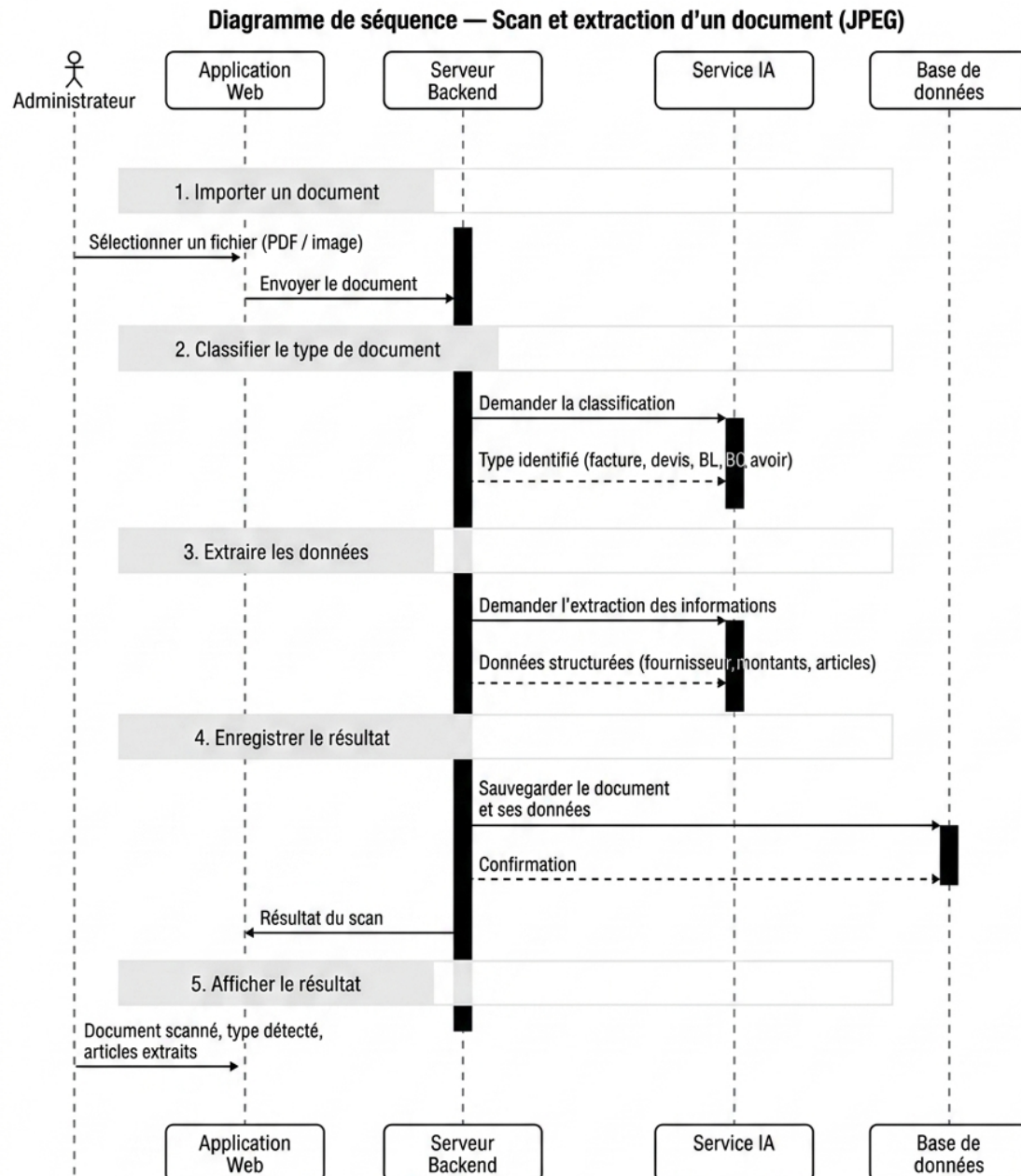


FIGURE 6. *Diagramme de séquence - Scan et extraction*

2.9.2 Processus de comparaison de prix

Cette figure 7 met en évidence le diagramme de séquence du processus de comparaison de prix, incluant l'extraction des articles depuis plusieurs documents, le calcul des prix totaux et l'identification des meilleures offres fournisseurs.

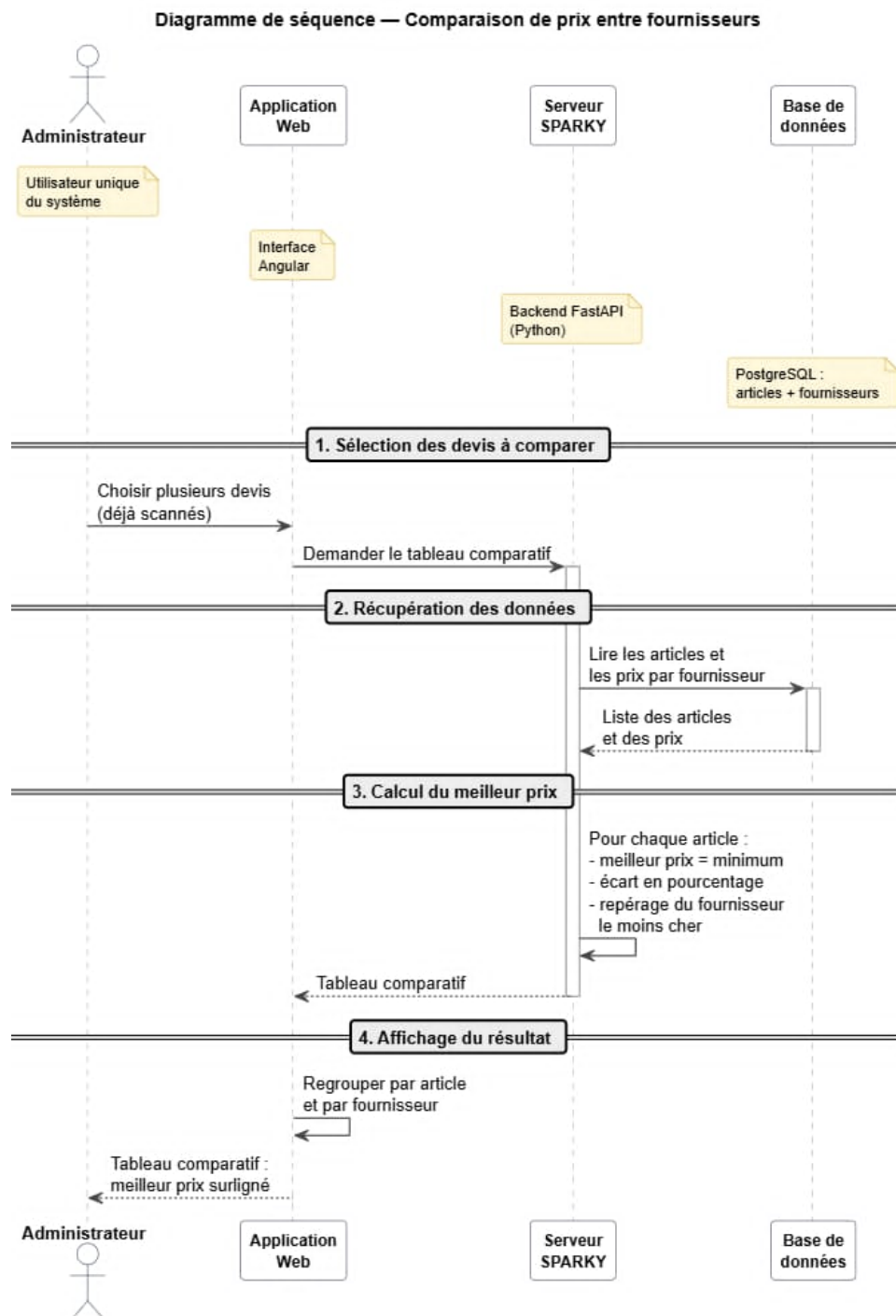


FIGURE 7. Diagramme de séquence - Comparaison de prix

2.10 *Choix technologiques*

Les choix technologiques resultent d'une analyse multicriteres combinant performance, maturite, communaute, documentation et adequation aux besoins.

2.10.1 Frontend : Angular 20

Angular 20 a été choisi pour sa performance de pointe, son architecture "Zoneless" native, et sa parfaite adéquation avec les standards technologiques les plus récents. La généralisation des Signals et l'introduction des Signal Forms optimisent la réactivité de l'application en offrant une gestion d'état plus fine et prévisible, éliminant ainsi les surcharges liées à l'ancienne détection de changements. L'écosystème enrichi et la sécurité renforcée garantissent une base robuste et pérenne, permettant de maintenir une interface moderne et performante tout en réduisant durablement la dette technique.

Cette figure 8 illustre le logo d'Angular utilisé dans le projet.



FIGURE 8. *Logo Angular*

2.10.2 Backend : FastAPI

FastAPI (version 0.115+) a été préféré à Flask et Django pour sa performance asynchrone native, sa génération automatique de documentation OpenAPI/Swagger, et son intégration native avec Pydantic v2 pour la validation des données. Uvicorn (version 0.30+), serveur ASGI sous-jacent, offre une capacité de traitement concurrent élevée, essentielle pour les appels IA qui peuvent être longs. Cette combinaison est particulièrement compatible avec Python 3.12.

Cette figure 9 illustre le framework FastAPI utilisé dans le backend du système Sparky.



FIGURE 9. *Logo Fast API*

2.10.3 IA : Mistral AI

Mistral AI a été sélectionnée pour la qualité de ses modèles de vision et de langage, sa compétitivité par rapport à OpenAI et Google, et son API bien documentée. Le modèle pixtral-12b-2409 est optimisé pour l'analyse d'images de documents, tandis que mistral-small-latest est utilisé pour les PDF textuels. L'API offre un bon compromis entre précision et latence.

Cette figure 10 illustre le logo de Mistral AI utilisé dans le système Sparky.



FIGURE 10. *Logo Mistral AI*

2.10.4 Base de données : PostgreSQL

PostgreSQL est le choix par défaut pour les applications nécessitant des garanties ACID, une richesse de types de données (JSONB, tableaux, géométrie), et une extensibilité via des extensions. Pour Sparky, la capacité à stocker des documents JSONB pour les résultats d'extraction flexibles constitue un atout majeur.

Dans ce projet, la version utilisée est PostgreSQL 16.13, avec le driver asyncpg 0.29.0 permettant une communication asynchrone performante et optimisée avec le backend, assurant ainsi une meilleure réactivité et scalabilité des opérations sur la base de données.

Cette figure 11 présente le logo de PostgreSQL, utilisé comme système de gestion de base de données dans le projet Sparky.



FIGURE 11. *Logo PostgreSQL*

2.10.5 Middleware : n8n

n8n, utilisé dans sa version 2.7.5, est une solution open-source d'automatisation de workflows, alternative à Zapier et Make. Elle permet un hébergement local, une bonne performance et la modification des workflows via un éditeur visuel sans redéploiement, offrant ainsi flexibilité et agilité opérationnelle.

Cette figure 12 présente le logo de n8n, outil d'orchestration des workflows utilisé dans l'architecture du système Sparky.



FIGURE 12. *Logo N8N*

2.11 Conclusion

Ce chapitre a présenté la conception architecturale du système Sparky basée sur une architecture en trois tiers, garantissant modularité et évolutivité. Il a également couvert les besoins fonctionnels et non fonctionnels du système, ainsi que les principaux choix techniques et la modélisation UML permettant de structurer l'ensemble de manière cohérente. Le chapitre suivant détaille la phase de développement et d'implémentation.

Chapitre

3

Développement et Implémentation

3.1 *Introduction*

La phase de développement constitue la matérialisation concrète des choix de conception élaborés au chapitre précédent. C'est durant cette étape que les spécifications deviennent code exécutable, que les architectures théoriques se transforment en systèmes opérationnels, et que les algorithmes abstraits prennent vie dans des interfaces interactives.

Ce chapitre détaille l'environnement de développement, l'implémentation du backend et du frontend, le rôle du middleware n8n, ainsi qu'une analyse comparative approfondie entre la première solution prototype et la solution finale retenue.

3.2 *Environnement de développement*

La mise en place d'un environnement de développement cohérent est un facteur déterminant pour garantir la qualité, la reproductibilité et la maintenabilité du code.

Le projet Sparky repose sur une architecture de dépôt de type monorepo structurée en quatre modules principaux :

- **frontend/** : application Angular 20
- **backend/** : API FastAPI

- **n8n/** : workflows d'automatisation
- **database/** : scripts PostgreSQL

L'environnement local repose sur :

- Node.js 20 pour le frontend
- Python 3.12 avec Uvicorn pour le backend
- Docker pour la conteneurisation de l'ensemble des services (frontend, backend, PostgreSQL et n8n)

3.2.1 Utilisation de Docker

Docker est au cœur de l'environnement de développement et de déploiement du projet Sparky. Il permet d'encapsuler chaque service dans un conteneur isolé, garantissant un fonctionnement identique quel que soit l'environnement (machine locale, serveur de test ou production).

Dans Sparky, Docker est utilisé pour conteneuriser les principaux services de l'application :

- **Base de données PostgreSQL** : exécutée dans un conteneur, avec un volume persistant pour assurer la sauvegarde des données.
- **n8n** : lancé via l'image officielle `n8nio/n8n`, avec un stockage persistant permettant de conserver les workflows. Son interface est accessible via le port 5678.
- **Backend Python** : conteneurisé avec Docker, il exécute l'application (via Uvicorn) et expose une API accessible aux autres services.
- **Frontend Angular** : également conteneurisé, il permet d'exécuter et servir l'interface utilisateur dans un environnement isolé et cohérent.

L'orchestration de l'ensemble est assurée par Docker Compose à l'aide du fichier `docker-compose.yml`.

Une seule commande permet de démarrer tous les services simultanément.

L'utilisation de Docker apporte plusieurs avantages :

- uniformité des environnements (développement, test, production),
- isolation des services,

- simplification du déploiement,
- meilleure gestion des dépendances.

Ainsi, Docker facilite considérablement le développement et rend le projet Sparky plus robuste et portable.

3.3 *Développement du backend*

3.3.1 Architecture générale

Le backend, développé en Python 3.12 avec FastAPI, constitue le cœur logique du système. Il expose une API REST documentée automatiquement via Swagger/OpenAPI.

L'architecture suit un modèle en couches afin de garantir la séparation des responsabilités :

- **main.py** : point d'entrée de l'application et définition des endpoints FastAPI
- **services** : logique métier du système
- **models.py** : définition et validation des structures de données
- **database.py** : gestion de la connexion et de la persistance des données
- **config.py** : configuration globale du système

3.3.2 Couche métier (Services)

La couche `services` regroupe l'ensemble de la logique métier du système Sparky et est organisée en modules spécialisés :

- `extractor.py` : extraction des informations depuis les documents via OCR et IA
- `classifier.py` : classification automatique des types de documents
- `comparator.py` : comparaison des prix entre fournisseurs
- `articles_db.py` : gestion des articles et interactions avec la base de données
- `normalizer.py` : normalisation et structuration des données extraites

3.3.3 Pipeline d'extraction IA

Le système implémente une stratégie d'extraction en cascade :

- Mistral AI (pixtral-12b / mistral-small)
- Fallback local (pdfplumber + regex)

Cette approche garantit :

- robustesse face aux échecs IA
- continuité de service
- adaptation selon le type de document

Cette figure 13 illustre le pipeline d'extraction basé sur l'intelligence artificielle et le mécanisme de fallback utilisé dans le système Sparky.

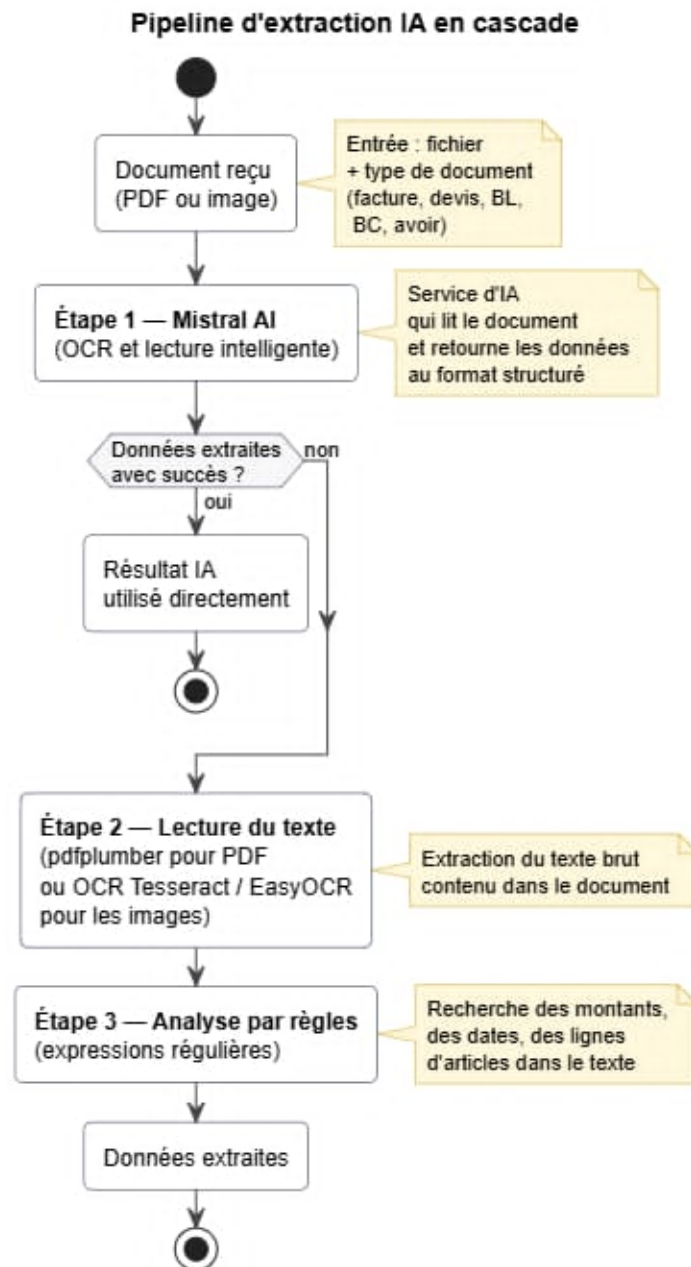


FIGURE 13. Pipeline d'extraction IA en cascade

3.3.4 Diagramme d'États — Cycle de Vie d'un Document

Le cycle de vie d'un document dans le système Sparky comporte quatre états principaux : PENDING, CLASSIFIED, EXTRACTED et ERROR. Après son upload, le document est d'abord en PENDING en attente de traitement. Il est ensuite classifié : s'il est correctement analysé, il passe à CLASSIFIED, sinon il est dirigé vers ERROR.

Ensuite, une extraction des données est effectuée ; si elle réussit, le document atteint l'état final EXTRACTED, sinon il passe également à ERROR. L'état ERROR regroupe donc tous les échecs et peut nécessiter une correction ou une nouvelle tentative de traitement.

Cette figure 14 illustre le diagramme d'états du cycle de vie d'un document dans le système Sparky.

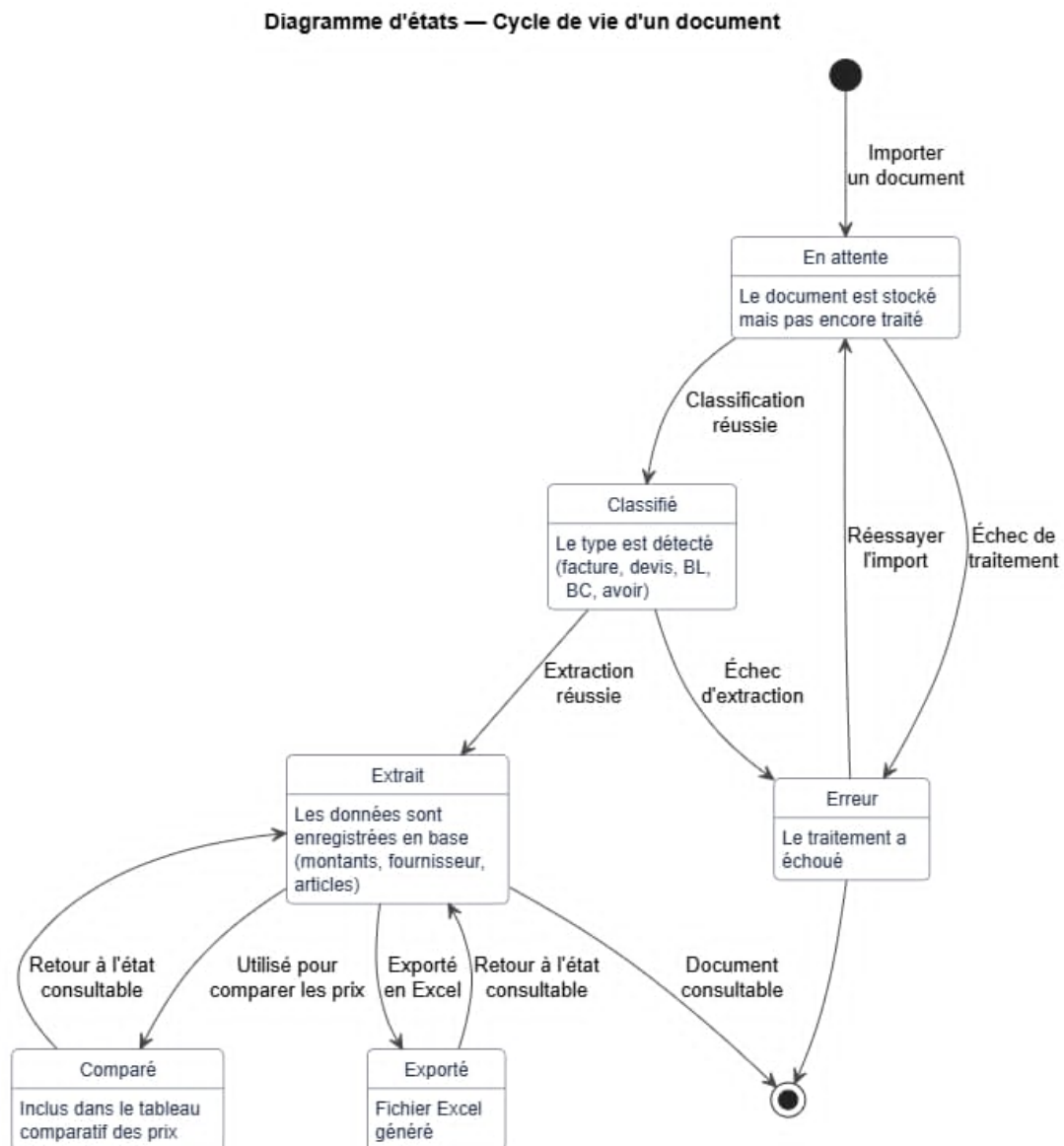


FIGURE 14. Diagramme d'états du cycle de vie d'un document

Le tableau 7 synthétise les différents états du cycle de vie d'un document ainsi que leurs transitions associées. Il permet de formaliser le déroulement du processus de traitement, depuis l'upload jusqu'à l'extraction finale ou la gestion des erreurs.

Cette modélisation améliore la traçabilité des opérations et facilite le suivi ainsi que la gestion des exceptions dans le pipeline d'extraction intelligent.

TABLEAU 7. *Description des états du cycle de vie*

État	Description	Transitions Possibles
PENDING	Document uploadé, stocké, en attente d'analyse	→ CLASSIFIED (succès), → ERROR (échec technique)
CLASSIFIED	Type identifié avec score de confiance	→ EXTRACTED (succès), → ERROR (échec extraction)
EXTRACTED	Données structurées extraites et sauvegardées	→ [Final], peut être reclassifié si erreur détectée
ERROR	Échec irréversible du traitement	→ PENDING (retry manuel), → Suppression

3.4 Développement du frontend

3.4.1 Architecture du Frontend

Le frontend est développé avec **Angular 20**, selon une architecture modulaire basée sur les **composants standalone**. Cette approche permet une meilleure organisation du code, une réutilisabilité accrue et une maintenance simplifiée.

L'organisation du projet est structurée comme suit :

- **components/** : regroupe les composants d'interface réutilisables, notamment la navbar et d'autres éléments UI génériques.
- **pages/** : contient les composants correspondant aux vues principales de l'application :
 - DashboardComponent
 - ScannerComponent
 - DocumentListComponent
 - PriceComparatorComponent
- **services/** : regroupe la logique métier et la communication avec l'API :
 - ApiService : gestion des requêtes HTTP
 - DocumentService : gestion de l'état des documents et stockage local
 - ExportService : export des données au format Excel
- **models/** : contient les interfaces de données, notamment `document.model.ts`.
- **environments/** : permet la configuration des environnements (développement et production) ainsi que des URLs d'API.

3.4.2 Gestion des services

Les services assurent la logique applicative côté client et l'interaction avec le backend :

- **ApiService** : centralise les requêtes HTTP et la communication avec l'API

- **DocumentService** : gère l'état des documents et leur stockage local
- **ExportService** : permet l'export des données au format Excel

3.4.3 Gestion de l'état

L'application utilise les **Signals Angular** pour la gestion réactive de l'état.

Cette approche permet :

- la centralisation de l'état des documents via `documentSignal`
- la mise à jour automatique de l'interface utilisateur
- une simplification par rapport à l'utilisation de RxJS

3.4.4 Interface utilisateur

L'interface repose sur **Angular Material**, garantissant une cohérence visuelle et une accessibilité conforme aux standards.

Les principaux composants utilisés sont :

- `MatTable`
- `MatCard`
- `MatButton`
- `MatSnackBar`
- `MatIcon`

3.5 *Middleware n8n*

Le middleware **n8n** joue un rôle d'orchestrateur central dans l'architecture de Sparky. Il assure la coordination des différentes étapes du pipeline de traitement documentaire à travers un workflow visuel entièrement configurable.

À chaque scan de document, le backend **FastAPI** déclenche automatiquement ce workflow via une requête HTTP POST envoyée au webhook n8n, incluant le fichier (PDF ou image) ainsi que les métadonnées associées (nom du fichier, horodatage et identifiant utilisateur).

3.5.1 Description du workflow n8n

Le workflow implémenté dans SPARKY est composé de onze nœuds séquentiels, décrits dans le Tableau 8.

L'avantage majeur de cette orchestration via n8n réside dans sa modifiabilité sans code. Un responsable métier peut modifier les destinataires des emails ou reconfigurer la structure de la feuille Google Sheets directement depuis l'interface visuelle de n8n, sans intervention du développeur.

Cette approche low-code complète efficacement la solution full-code basée sur FastAPI.

TABLEAU 8. *Tableau des nœuds n8n — SPARKY*

#	Nœud	Type	Rôle
1	Réception Document	webhook	Reçoit les données envoyées par FastAPI après traitement (classification + extraction déjà faites)
2	Validation et Détection Type	code	Détecte le type de fichier (image/PDF) et structure les données pour les nœuds suivants
3	Labellisation Type Document	code	Formate le score de confiance (0.92 → 92%) et calcule le niveau : Élevé $\geq 85\%$, Moyen $\geq 60\%$, Faible $< 60\%$
4	Résultat Classification	code	Log de contrôle — affiche le type et le score dans la console n8n
5	Calcul Total & Métadonnées Modèle	code	Calcule le total TTC en additionnant les lignes d'articles et identifie le modèle Mistral utilisé
6	Résultat Extraction	code	Log de contrôle — affiche le nombre d'articles et le moteur IA dans la console n8n
7	Préparer lignes Google Sheet	code	Transforme 1 document avec N articles en N lignes séparées (1 article = 1 ligne)
8	Export Google Sheet	googleSheets	Insère automatiquement chaque ligne d'article dans la feuille Google Sheets partagée
9	Construire Email HTML	code	Génère le corps HTML complet de l'email : tableau articles, montants, badge de confiance, bouton Google Sheets
10	Envoyer Email	emailSend	Envoie l'email de notification via SMTP Gmail aux destinataires configurés
11	Réponse Finale	respondToWebhook	Retourne une confirmation JSON à FastAPI avec le statut, le nombre d'articles et le lien Google Sheets

La Figure 15 illustre la conception et l'exécution d'un workflow dans n8n.

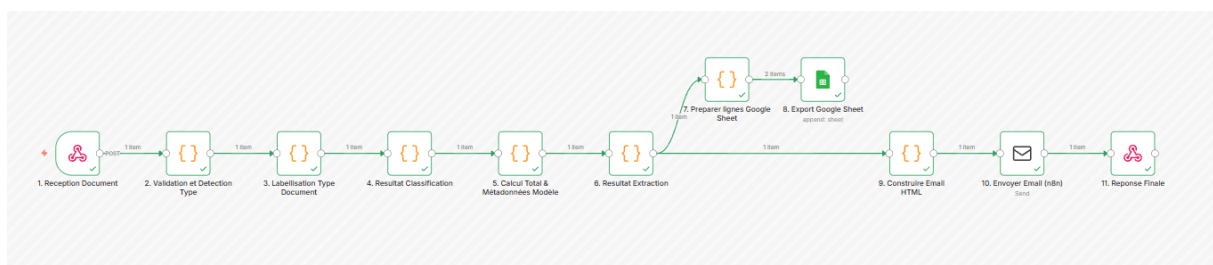


FIGURE 15. *Conception et exécution d'un workflow avec n8n*

3.5.2 Exemple de notification email

Le workflow automatisé dans n8n génère une notification email complète après le traitement et l'extraction des données d'un document, comme illustré dans la figure 16.

Cet email constitue une synthèse structurée et lisible des informations extraites par le système d'intelligence artificielle. Il est composé d'un en-tête contenant le logo SPARKY, suivi d'une section de classification IA indiquant le type de document détecté ainsi que le niveau de confiance du modèle. L'email présente ensuite les informations principales du document, notamment le fournisseur, le numéro de document, la date, ainsi que les montants HT, TVA et TTC mis en évidence pour faciliter la lecture.

Une section dédiée aux articles extraits est également intégrée sous forme de tableau structuré, incluant la désignation, la référence, la quantité, le prix unitaire, la remise, la TVA et le total TTC.

Enfin, un bouton d'action vert "Voir dans Google Sheets →" permet à l'utilisateur d'accéder directement à la feuille de calcul contenant l'ensemble des données structurées, chaque article étant enregistré sur une ligne distincte.

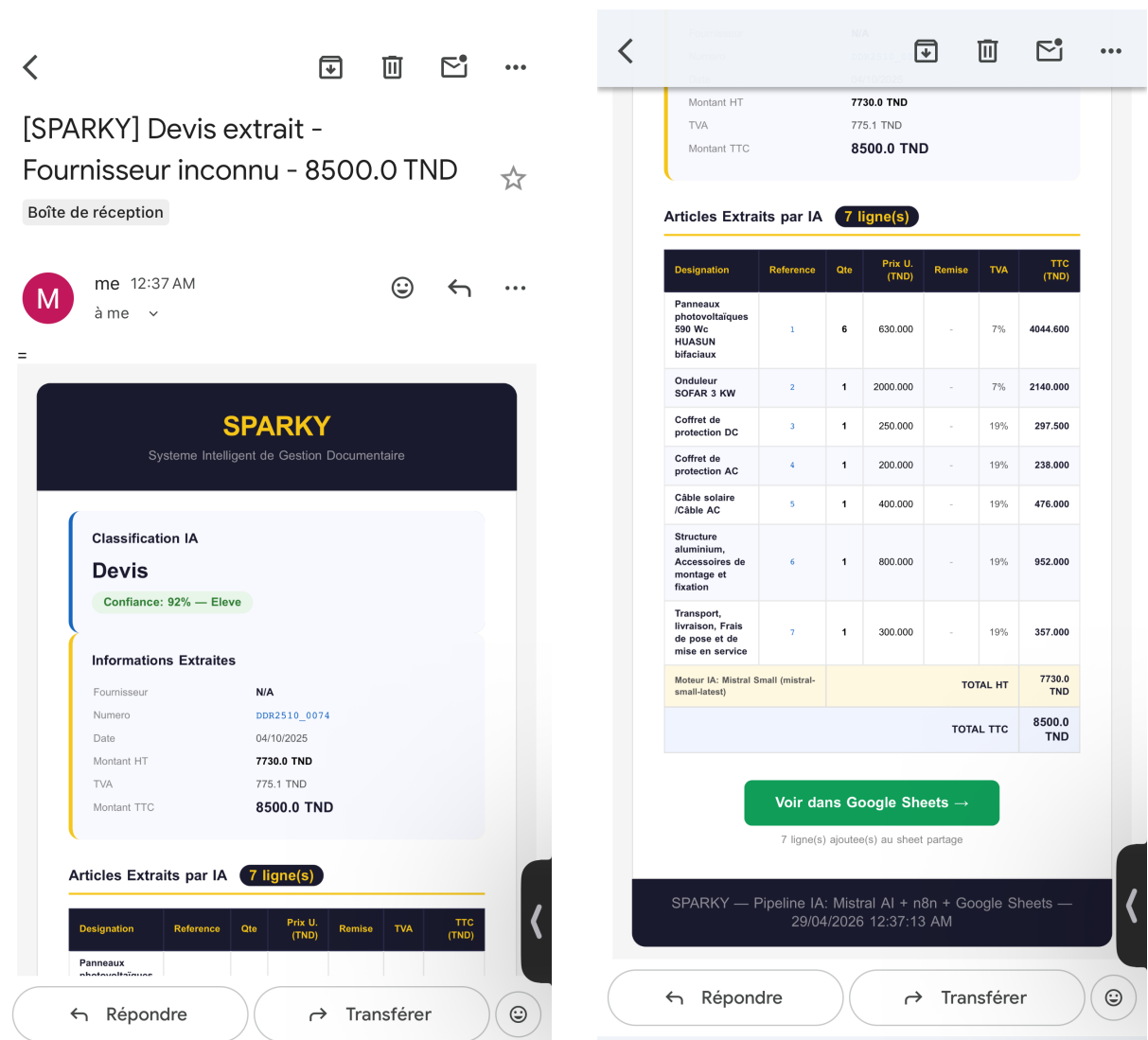


FIGURE 16. Exemple de notification email générée par le workflow SPARKY

Les données extraites sont ensuite automatiquement enregistrées dans une feuille de calcul Google Sheets, où chaque ligne correspond à un article du document traité. Cette organisation permet une exploitation simple et structurée des informations, facilitant le suivi, l’analyse et l’intégration avec d’autres systèmes métiers. L’interface du tableau de bord illustre clairement cette structuration des données, avec des colonnes dédiées aux désignations, références, quantités et prix.

Cette figure 17 présente un exemple de données structurées enregistrées automatiquement dans Google Sheets après traitement des documents.

A	B	C	D	E	F	G	H	I	J	K	L	M	N
document_id	type_document	fournisseur	numeros_document	date_document	description	reference	quantite	prix_unitaire	remise	tva	montant_ht_ligne	montant_ht_total	montant
1	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Vis à bois 4x40 mm (boîte 200 pcs)	QNC-001	20	12,5	0	19	250	250	250
2	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Charnière acier zingué 60mm	QNC-002	50	3,2	0	19	160	160	160
3	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Serrure encastrée 3 points	QNC-003	10	185	0	19	1850	1850	1850
4	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Poignée de porte aluminium (paire)	QNC-004	15	42	0	19	630	630	630
5	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Câble acier galvanisé 05mm (rouleau 50m)	QNC-005	5	320	0	19	1600	1600	1600
6	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Bouton M10x50 - acier (boîte 50 pcs)	QNC-006	30	15,75	0	19	472,5	472,5	472,5
7	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Ruban adhésif aluminium 50mm x 50m	QNC-007	40	8,9	0	19	356	356	356
8	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Ancre chimique résine époxy 300ml	QNC-008	25	28	0	19	700	700	700
9	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Lame de scie circulaire Ø190mm 40 dents	QNC-009	8	54	0	19	432	432	432
10	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Vernis de sécurité cadenas 60mm	QNC-010	20	35	0	19	700	700	700
11	2025-04-28722.36.57.0712	Davis	SOTUFER - Fournitures	0504/2025	Vis à bois 4x40 mm (boîte 200 pcs)	QNC-001	20	12,5	0	19	250	250	250
12	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Charnière acier zingué 60mm	QNC-002	50	3,2	0	19	160	160	160
13	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Serrure encastrée 3 points	QNC-003	10	182	0	19	1820	1820	1820
14	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Poignée de porte aluminium (paire)	QNC-004	15	40,5	0	19	607,5	607,5	607,5
15	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Câble acier galvanisé 05mm (rouleau 50m)	QNC-005	5	312	0	19	1560	1560	1560
16	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Bouton M10x50 - acier (boîte 50 pcs)	QNC-006	30	15	0	19	450	450	450
17	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Ruban adhésif aluminium 50mm x 50m	QNC-007	40	8,6	0	19	344	344	344
18	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Ancre chimique résine époxy 300ml	QNC-008	25	27	0	19	675	675	675
19	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Lame de scie circulaire Ø190mm 40 dents	QNC-009	8	52,5	0	19	420	420	420
20	2025-04-28722.42.31.6232	Davis	ELBENNA MATERIEL & OUTILLAGE	0704/2025	Vernis de sécurité cadenas 60mm	QNC-010	20	33,5	0	19	670	670	670
21	2025-04-28723.33.45.0302	Davis	N/A		(aucune ligne)		0	0	0	0	0	0	0
22	2025-04-28723.33.45.0302	Davis	N/A				0	0	0	0	0	0	0
23	2025-04-28723.35.17.4112	Facture		30/09/2025	OASOL		629 723	1,688	0	13	1201,91		
24	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Panneaux photovoltaïques 590 Wc HUASUN bifaciaux	1	6	630	0	7	4044,6		
25	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Onduleur SOFAR 3 KW	2	1	2000	0	7	2140		
26	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Coffret de protection DC	3	1	250	0	19	297,5		
27	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Coffret de protection AC	4	1	200	0	19	238		
28	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Câble solaire Câble AC	5	1	400	0	19	479		
29	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Structure aluminium, Accessoires de montage et fixation	6	1	800	0	19	952		
30	2025-04-28723.37.10.1732	Davis	N/A	04/10/2025	Transport, livraison, Frais de pose et de mise en service	7	1	300	0	19	357		

FIGURE 17. Exemple de données structurées dans Google Sheets

3.6 Première solution : $n8n$ + Ollama Qwen2.5-B7

Une première version du système a été développée autour d’une architecture entièrement locale.

Flux :

- réception document via $n8n$
- OCR via Mistral vision
- extraction via Qwen2.5-B7 (Ollama)
- stockage Google Sheets

3.6.1 Avantages

- confidentialité totale (local)
- coût nul
- bonne précision d’extraction

3.6.2 Limites

- temps de réponse élevé (30 à 120s)
- absence de scalabilité
- gestion d’erreurs limitée

3.7 Comparaison des solutions

TABEAU 9. *Comparaison comparative des deux solutions*

Critère	Solution n8n + Ollama	Solution FastAPI + Mistral	Observation
Temps de réponse	120-180s / document	3-8s / document	Gain de 90%
Confidentialité	Excellente (local)	Bonne (API sécurisée)	Légère dégradation
Coût opérationnel	Nul	Fonction du volume	Compromis acceptable
Précision OCR	Très bonne	Excellente	Amélioration
Scalabilité	Faible (séquentiel)	Élevée (async)	Décisif
Base de données	Google Sheets	PostgreSQL	Professionnalisation
Interface utilisateur	Limitée (n8n)	Complète (Angular)	Déterminant
Maintenabilité	Moyenne	Excellente	Code versionné
Export de données	CSV basique	Excel formaté + CSV	Enrichi

Le choix de la solution finale repose principalement sur le **temps de réponse**, qui a été fortement amélioré grâce à l'association FastAPI + Mistral AI, réduisant le temps de traitement à moins de 10 secondes, contre 180 secondes pour la solution prototype.

Le critère de **confidentialité** demeure satisfaisant malgré l'utilisation d'une API externe, grâce aux garanties offertes par Mistral AI (non-conservation des données et conformité au RGPD). Un mode de traitement local basé sur pdfplumber et les expressions régulières reste disponible pour les documents sensibles.

Le passage de **Google Sheets à PostgreSQL** constitue une amélioration majeure, permettant une meilleure modélisation relationnelle, des performances accrues, une sécurité renforcée ainsi que des capacités avancées de reporting et d'analyse.

Enfin, le développement d'une interface avec **Angular** améliore significativement l'expérience utilisateur, en offrant une navigation fluide, des retours visuels immédiats, des tableaux interactifs et un tableau de bord analytique, transformant ainsi la solution en une application complète et professionnelle.

3.8 Choix final et justification

La solution finale repose sur un compromis entre performance, maintenabilité et expérience utilisateur.

Les critères déterminants sont :

- le temps de réponse en contexte réel
- une expérience utilisateur fluide
- une architecture scalable
- une maintenabilité simplifiée du système

3.9 Conclusion

Ce chapitre a présenté l'intégralité du processus de développement du système Sparky. Nous avons détaillé l'environnement de développement, l'implémentation du backend FastAPI avec son pipeline d'extraction IA en cascade, le frontend Angular 20 basé sur les Signals, et le rôle du middleware n8n. Le cœur de ce chapitre réside dans l'analyse comparative entre la solution prototype basée sur Ollama en local et la solution finale basée sur Mistral AI en API.

Cette comparaison, fondée sur des critères objectifs de performance, sécurité, coût et maintenabilité, a démontré l'importance d'évaluer chaque alternative dans son contexte d'usage. Le chapitre suivant présentera l'application déployée et ses interfaces utilisateur.

Chapitre

4

Présentation de l'Application

4.1 *Introduction*

Ce chapitre concrétise les phases précédentes en présentant l'application Sparky telle qu'elle est déployée et utilisée.

Il décrit les principales interfaces utilisateur, leurs fonctionnalités, ainsi que la manière dont elles répondent aux besoins identifiés au chapitre 2.

Les captures d'écran illustrent l'expérience utilisateur finale et permettent de valider la conformité de l'implémentation avec les spécifications définies.

4.2 *Page de connexion*

La page de connexion constitue le point d'entrée sécurisé de l'application Sparky. Elle permet l'authentification des utilisateurs avant l'accès aux fonctionnalités principales du système.

L'interface est volontairement simple et épurée afin de faciliter l'expérience utilisateur. Elle est composée de deux champs principaux : l'identifiant (ou email) et le mot de passe, ainsi que d'un bouton de connexion permettant de valider les informations saisies.

Une fois les informations authentifiées avec succès, l'utilisateur est redirigé vers le module Scanner

En cas d'erreur d'authentification, un message d'erreur est affiché afin d'informer l'utilisateur que les identifiants sont incorrects.

Cette figure 18 présente l'interface de la page de connexion du système Sparky.

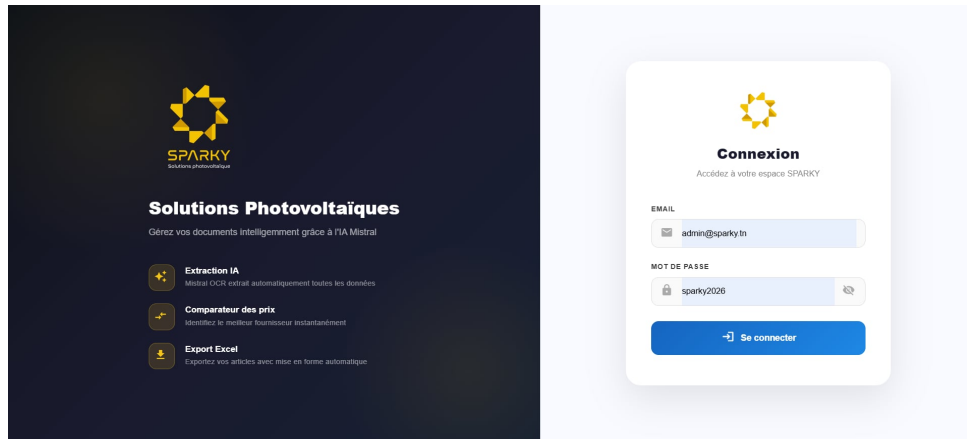


FIGURE 18. *Interface de la page de connexion*

4.3 Module Tableau de bord

Le tableau de bord constitue la page d'accueil de l'application Sparky. Il offre à l'utilisateur une vision synthétique et instantanée de l'activité documentaire de son entreprise via une interface claire et structurée, comme présenté dans la figure 19.

En haut de page, une bannière d'accueil affiche le nom de l'application accompagné de son positionnement « Gestion intelligente des documents photovoltaïques », avec un accès direct au scanner via un bouton dédié. La zone centrale présente six compteurs statistiques en temps réel, répartis par type de document : total des documents traités, factures, bons de livraison, bons de commande, avoirs et devis. Chaque compteur est associé à une icône et un code couleur distinctifs, permettant une lecture rapide de la répartition du portefeuille documentaire.

En bas de page, une section « Actions rapides » regroupe trois raccourcis illustrés vers les fonctionnalités principales : Scanner un document, Comparateur des prix et Voir les documents. Ces trois actions couvrent l'intégralité du flux de travail de l'utilisateur et garantissent un accès en un seul clic aux modules essentiels de l'application.

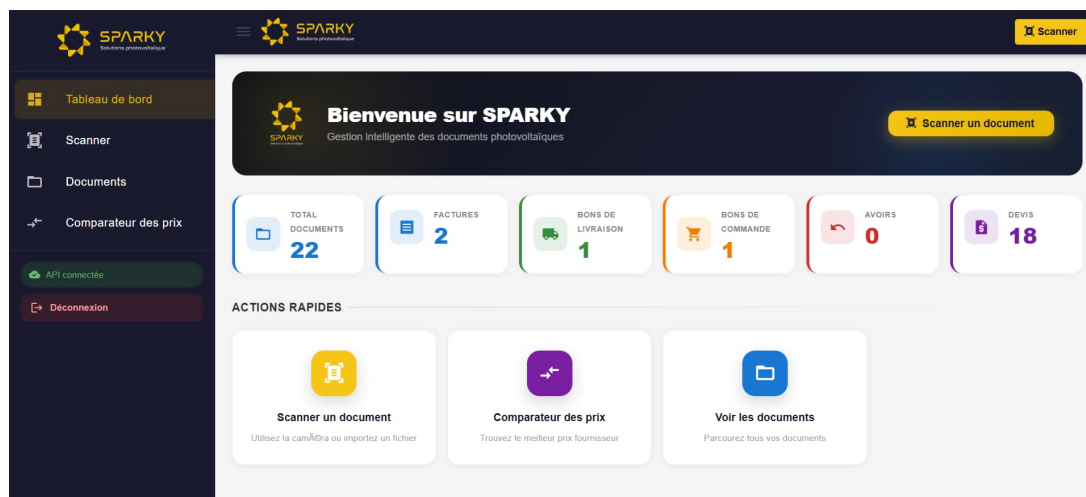


FIGURE 19. Interface du module Tableau de bord

4.4 Module Scanner de documents

Le module Scanner constitue le point d'entrée principal de l'application Sparky. Il assure la numérisation ainsi que le traitement automatisé des documents au sein d'une interface à la fois simple et intuitive. L'utilisateur peut importer un fichier soit par glisser-déposer, soit via un bouton de sélection classique. Une prévisualisation du document est ensuite affichée, accompagnée d'un indicateur de progression illustrant le traitement par intelligence artificielle, comme présenté dans la figure 20.

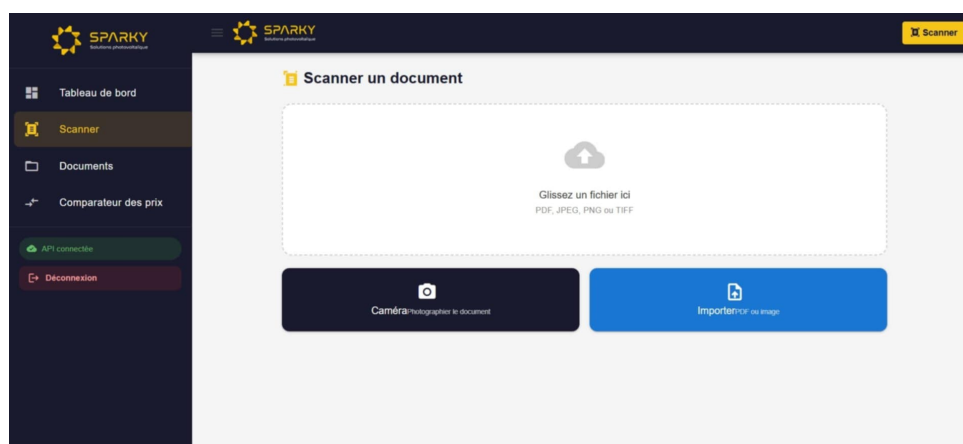


FIGURE 20. Interface du module Scanner de documents

CHAPITRE 4. PRÉSENTATION DE L'APPLICATION

À l'issue du traitement, les informations extraites sont structurées sous forme de tableau regroupant les principales données du document, notamment le fournisseur, le numéro, la date, les montants ainsi que les détails des articles, comme présenté dans la figure 21.

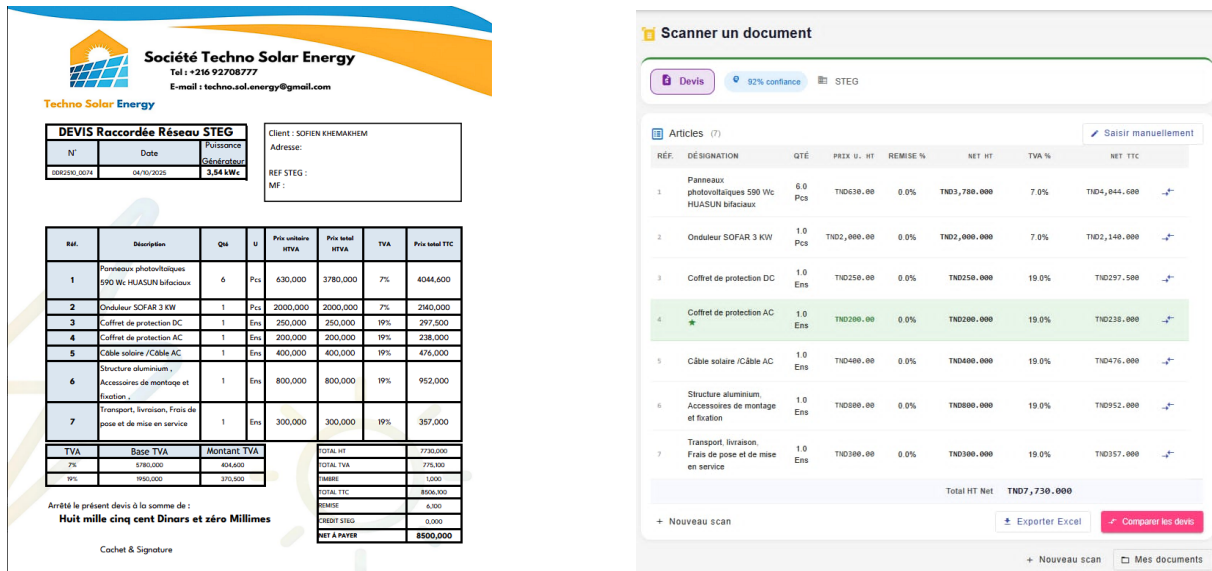


FIGURE 21. Présentation structurée des informations extraites à partir d'un devis

Un module d'export permet enfin de générer un fichier Excel exploitable dans les systèmes comptables.

4.5 Module Comparateur de prix

Le module Comparateur de prix constitue une fonctionnalité avancée de Sparky, permettant l'optimisation des décisions d'achat.

L'utilisateur peut importer plusieurs devis ou factures de fournisseurs via une interface de téléchargement unique. Le système extrait automatiquement les données des documents sans intervention manuelle.

Les résultats sont présentés sous forme d'un tableau comparatif où les lignes représentent les articles et les colonnes les fournisseurs. Les prix unitaires sont affichés pour chaque combinaison article-fournisseur, facilitant la comparaison directe des offres. Le meilleur prix pour chaque article est mis en évidence afin de faciliter la prise de décision. Un calcul automatique des totaux par fournisseur est également effectué, permettant d'identifier rapidement l'offre la plus avantageuse.

Cette figure 22 illustre l'interface du module Comparateur de prix.

Figure 22 shows the 'Comparateur des prix' (Price Comparator) interface. It displays a comparison of prices for 10 items across three suppliers: MAGHREB QUINCAILLERIE, ELBENNA MATERIEL & OUTILLAGE, and SOTUFER - Fournitures. The table below summarizes the data shown in the interface.

Article / Désignation	Stock	MAGHREB QUINCAILLERIE	ELBENNA MATERIEL & OUTILLAGE	SOTUFER - Fournitures
vis à bois 4x40 mm (boîte 200 pcs)	—	11.800 TND x 20	12.100 TND x 20	12.500 TND x 20
charnière acier zingué 60mm	—	3.500 TND x 50	3.000 TND x 50	3.200 TND x 50
serrure encastrée 3 points	—	178.500 TND x 10	182.000 TND x 10	185.000 TND x 10
câble acier galvanisé ø5mm (rouleau 50m)	—	305.000 TND x 5	312.000 TND x 5	320.000 TND x 5
boulon m10x60 + écrou (boîte 50 pcs)	—	19.200 TND x 30	18.000 TND x 30	18.750 TND x 30
ruban adhésif aluminium 50mm x 50m	—	8.400 TND x 40	8.600 TND x 40	8.900 TND x 40
ancrage chimique résine époxy 300ml	—	26.500 TND x 25	27.000 TND x 25	28.000 TND x 25
lame de scie circulaire ø190mm 40 dents	—	58.000 TND x 8	52.500 TND x 8	54.000 TND x 8
verrou de sécurité cadenas 60mm	—	32.000 TND x 20	33.500 TND x 20	35.000 TND x 20
Total TTC	—	7,074.50 TND	7,028.50 TND	7,240.50 TND

FIGURE 22. Interface du module Comparateur de prix

4.6 Matching IA + PostgreSQL

Au cœur du comparateur de prix se trouve un algorithme de normalisation et de matching intelligent des articles, implémenté dans le service `ProductNormalizer` (`backend/services/normalizer.py`). Contrairement aux approches classiques basées sur des bibliothèques externes, ce module est entièrement développé en Python pur afin de garantir la portabilité et le contrôle total de la logique métier.

Le système résout un problème fondamental : un même article peut être désigné différemment selon les fournisseurs (variations lexicales, unités, formats ou références internes).

4.6.1 Stratégie de matching en cascade

L'algorithme repose sur une stratégie progressive à quatre niveaux, s'arrêtant dès qu'une correspondance fiable est trouvée :

- **1. Match exact par référence fournisseur** : si la `reference_fournisseur` existe déjà en base, le score est fixé à 1.0.
- **2. Match par spécifications techniques** : extraction des caractéristiques techniques (catégorie, dimensions, section, puissance, intensité, etc.), puis comparaison via une fonction dédiée `specs_similarity()` donnant un score entre 0.85 et 0.99.
- **3. Similarité textuelle par trigrammes** : utilisation d'une fonction maison `_trigram_similarity()` qui découpe les désignations en triplets de caractères et calcule un indice de Jaccard :

$$\text{similarité} = \frac{|A \cap B|}{|A \cup B|}$$

- **4. Aucun match** : création d'un nouvel article canonique dans la base.

4.6.2 Détection de catégorie

La classification des articles est réalisée via une détection basée sur des mots-clés métier (câbles, disjoncteurs, panneaux photovoltaïques, structures métalliques, etc.), avec une logique de priorité implémentée dans `normalizer.py`.

4.6.3 Stockage des données

Le système repose sur deux tables principales :

- **articles** : catalogue canonique unifié
- **articles_fournisseurs** : variantes fournisseurs avec score de similarité

Aucune dépendance externe n'est utilisée pour le matching (ni RapidFuzz, ni difflib, ni extensions PostgreSQL côté Python), ce qui garantit une solution légère et totalement maîtrisée.

4.6.4 Impact fonctionnel

Ce système de normalisation améliore fortement la qualité du comparateur de prix en automatisant la fusion des articles équivalents.

À titre comparatif, le traitement manuel d'un appel d'offres impliquant trois fournisseurs et dix articles nécessite en moyenne vingt minutes : ouverture et lecture des documents (3 min), saisie manuelle des données dans un tableau Excel (9 min), mise en forme et formules de comparaison (5 min), et vérification des erreurs de saisie (3 min). Avec Sparky, ce même traitement est réalisé en moins d'une minute : upload des fichiers (30 s), traitement IA (25 s) et accès au comparateur (10 s). Cela représente une réduction estimée de 95% du temps opérationnel, tout en éliminant les erreurs humaines liées à la saisie manuelle.

De plus, le système est évolutif : l'enrichissement progressif des données améliore la robustesse du matching et la précision des comparaisons au fil du temps.

4.7 Module Gestion des documents

Le module de gestion des documents permet la visualisation de l'ensemble des documents traités sous forme de cartes organisées en grille. Une codification par couleur est utilisée afin de distinguer les différents types de documents : orange pour les bons de commande, vert pour les bons de livraison, bleu pour les factures et violet pour les devis.

Chaque carte affiche les informations essentielles du document, notamment le nom du fichier, la référence, le fournisseur ainsi que le montant TTC en dinars tunisiens (TND). Elle intègre également un badge « Extracted » accompagné d'un score de confiance de 92 %, reflétant la qualité du processus d'extraction basé sur l'intelligence artificielle.

En partie supérieure de l'interface, une barre de filtrage permet de trier les documents selon leur type, complétée par un champ de recherche et une fonctionnalité de tri par date. Par ailleurs, la barre latérale regroupe les quatre modules principaux de l'application et affiche en temps réel l'état de connexion à l'API Mistral.

Cette figure 23 illustre l'interface du module de gestion des documents.

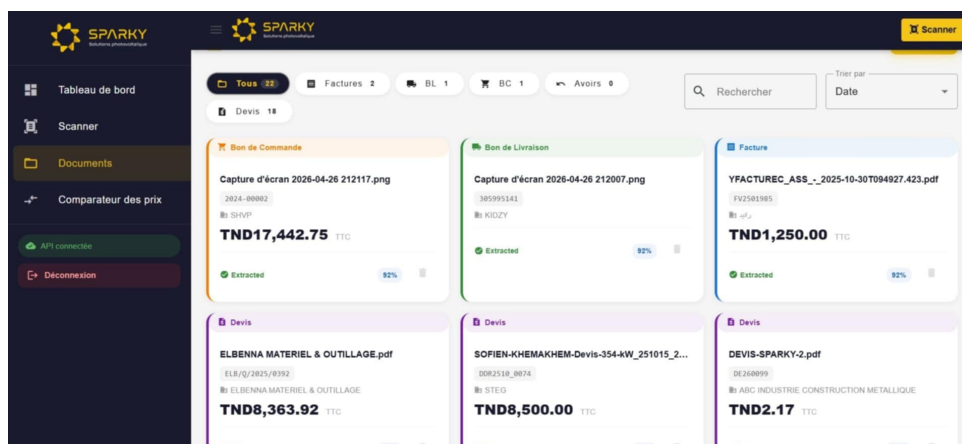


FIGURE 23. Interface du module Gestion des documents

4.8 Conclusion

Ce chapitre a présenté l'application Sparky dans son ensemble à travers ses principales interfaces utilisateur et ses modules fonctionnels. Chaque composant, de la page de connexion au module de gestion des documents, illustre la mise en œuvre concrète des différentes phases du système, notamment l'extraction, le traitement intelligent des documents et le comparateur de prix.

L'étude des interfaces met en évidence une application intuitive, structurée et orientée utilisateur, permettant de simplifier des tâches complexes telles que la comparaison des offres fournisseurs et la gestion documentaire. L'intégration de mécanismes d'intelligence artificielle, notamment pour le matching des articles et l'extraction des données, assure une automatisation efficace et une amélioration significative de la productivité.

Ainsi, l'application Sparky répond aux objectifs fixés en offrant une solution complète de gestion intelligente des documents photovoltaïques, tout en garantissant rapidité, précision et fiabilité dans le traitement des informations.



Conclusion générale

Le projet Sparky a permis de concevoir et de développer une application web complète dédiée au traitement intelligent des documents commerciaux. Ce travail a démontré la faisabilité technique ainsi que la valeur opérationnelle d'une solution intégrant des technologies modernes en intelligence artificielle, développement web et architecture distribuée.

Sur le plan fonctionnel, l'application répond aux principaux besoins identifiés : la numérisation et l'extraction automatique des documents via un moteur d'IA, la gestion centralisée et structurée des données, la comparaison intelligente des prix entre fournisseurs, le suivi historique des prix, ainsi que la visualisation analytique à travers un tableau de bord interactif. Chaque module a été conçu et validé conformément aux spécifications définies.

Sur le plan technique, l'architecture adoptée (Angular 20 / n8n / FastAPI) assure une séparation claire des responsabilités et une bonne évolutivité du système. FastAPI garantit des performances élevées pour les traitements asynchrones et les appels IA, tandis que Angular offre une interface réactive et structurée. De son côté, PostgreSQL assure la robustesse et la cohérence des données.

L'analyse comparative entre la solution prototype et la solution finale a mis en évidence un compromis essentiel entre confidentialité, performance et rapidité d'exécution. Cette démarche comparative constitue une approche d'ingénierie rigoureuse et pertinente pour guider les choix architecturaux.

En définitive, le projet Sparky illustre la capacité des technologies modernes à transformer un processus documentaire complexe en une solution automatisée, efficace et intuitive. Il met

également en évidence l'importance des choix technologiques dans la conception d'un système intelligent, où chaque décision résulte d'un équilibre entre performance, sécurité et évolutivité.



Webographie

- [1] *Angular Documentation*. URL : <https://angular.dev>.
- [2] *Node.js Documentation*. URL : <https://nodejs.org/docs>.
- [3] *FastAPI Documentation*. URL : <https://fastapi.tiangolo.com>.
- [4] *PostgreSQL Documentation*. URL : <https://www.postgresql.org/docs>.
- [5] *Docker Documentation*. URL : <https://docs.docker.com>.
- [6] *SQLAlchemy Documentation*. URL : <https://docs.sqlalchemy.org>.
- [7] *Mistral AI Platform*. URL : <https://docs.mistral.ai>.
- [8] *Mistral Vision (Pixtral-12B)*. URL : <https://docs.mistral.ai/capabilities/vision>.
- [9] *n8n Workflow Automation*. URL : <https://docs.n8n.io>.
- [10] *Google Sheets API*. URL : <https://developers.google.com/sheets/api>.
- [11] *Google Drive API*. URL : <https://developers.google.com/drive/api>.
- [12] *TF-IDF Feature Extraction - Scikit-learn*. URL : https://scikit-learn.org/stable/modules/feature_extraction.html.



Bibliographie

- [13] Andrew S. TANENBAUM. *Computer Networks*. Pearson, 2010.
- [14] Ian GOODFELLOW, Yoshua BENGIO et Aaron COURVILLE. *Deep Learning*. MIT Press, 2016.
- [15] Sam NEWMAN. *Building Microservices*. O'Reilly Media, 2021.
- [16] Nigel POULTON. *Docker Deep Dive*. Packt Publishing, 2023.
- [17] Ashbindu SINGH. "Review article digital change detection techniques using remotely-sensed data". In : *International journal of remote sensing* 10.6 (1989), p. 989-1003.